Ephemeral Interfaces: Design Patterns for Task-Scoped Generative UI Components

Daniel Kumlin

IE University

Capstone Project

Supervisor: Prof. *Jose Antonio Garcia Escandón*

Date (*April 2026*)

Acknowledgements

I would like to thank everyone who has helped guid me through the process of creating this thesis. It was a personal idea I really wanted to explore and felt empowered to try this with all the positive feedback I got from friends and colleagues.

I would like to thank my supervisor, Prof. Jose Antonio Garcia Escandón, for his guidance, feedback, and support throughout this capstone project.

I am also very grateful to the particpants who conducted the study. It was an interesting twist to traditional surveys so I really appreciate everyone who took the time to take it.

Finally thank you for my family and friends who have supported me along the way.

And with that, *Let the capstone begin.*

Abstract

Generative user interfaces have become an active topic in human-computer interaction and AI-supported software design because they promise interfaces that adapt to user goals instead of remaining fully static. Yet this promise creates a practical tension for productivity software, where users depend on stable layouts, predictable controls, and durable mental models. This thesis investigates whether ephemeral, task-scoped generative UI components can offer a bounded alternative to full interface regeneration by appearing only when locally useful and disappearing without destabilising the host application.

The thesis develops and tests a framework for ephemeral support through a web-based research artifact. The main individual contributions are the framework operationalisation itself, a constrained component-specification model for temporary support, a validation pipeline for generated interface specifications, a bounded ephemeral overlay approach that preserves interface continuity, and an instrumented comparative study artifact for empirical evaluation. The artifact was tested in a within-subjects user study comparing a baseline interface with an ephemeral condition on a bounded slide-reordering task.

The results show a mixed outcome. The ephemeral condition produced a slightly higher task completion rate and significantly higher helpfulness ratings, while perceived control remained stable, but it did not improve completion time or overall preference and significantly increased interaction count and perceived intrusiveness. These findings suggest that ephemeral generative support is most credible not as a general replacement for existing interfaces, but as a bounded design strategy for low-risk productivity tasks where lightweight contextual guidance may be useful. The thesis therefore contributes both a concrete design framework and an empirical basis for assessing when temporary generative assistance can add value without disrupting the user experience.

Keywords: generative UI, ephemeral interfaces, human-computer interaction, productivity software, design science research

Contents

1 Introduction

1.1 Problem and Motivation

Large language models are no longer confined to generating textual content; they are increasingly being used to produce user interfaces that adapt to the goals and context of individual users. This shift has opened a design space in which the structure of an application can respond to what a user needs rather than remaining fixed by its developers. Yet the promise of adaptive, AI-generated interfaces introduces a fundamental tension. On the one hand, static interfaces constrain users to predetermined layouts and interaction paths that may not match the diversity of real-world tasks. On the other hand, fully generative approaches that replace the entire interface risk undermining the predictability and continuity on which productive use depends.

This tension is especially acute in productivity software, where users build durable mental models of how their tools behave. When an interface is regenerated wholesale in response to each new request, orientation is lost: users must relearn the layout, relocate controls, and re-establish their sense of place within the application. The design challenge, therefore, is not simply whether AI can generate useful interfaces, but how generative capability can be introduced into an existing application without eroding the stability that makes it usable in the first place.

A promising middle ground lies in *ephemerality*: rather than replacing an interface entirely, the system could surface temporary, task-scoped components that appear only when contextually justified and disappear once their purpose has been served. Such components would augment the host application at the point of need while leaving the surrounding interface intact. Although this idea draws on established principles from both generative UI research and interaction design theory, a coherent framework that specifies when ephemeral components should appear, how their scope should be bounded, and how they should be withdrawn without disrupting the user's workflow has not been articulated in prior work.

1.2 Literature Review

While LLMs have been primarily used to generate content through text, they are increasingly being used to generate user interfaces (UI). It is a way to move beyond static interfaces by generating structures based on a set of primitives tailored to the users' goals. Prior work suggests that when working with LLMs, users prefer to express themselves through UI instead of a plain chat. This is because the system can present its functionality properly instead of forcing it through text (Ahmed & Imran, 2025; Leviathan et al., 2025; Tang et al., 2025). However, most of the evidence comes from the context of a fully generated UI, which differs from the design challenge addressed in this thesis, which is to augment existing applications where users expect a certain degree of stability and predictability in their tools. This raises the question of how a set of core primitive UI components can be integrated into already established workflows instead of replacing the entire state of the interface with AI.

The problem with generative UI interfaces is that while they can increase the relevance of the task at hand, they can threaten predictability and user control. In real-world domains such as banking, their rigid interfaces were built on a set of predefined rules that fail to meet the ever-changing needs of their users across services (Tambi, 2020). Approaches that dynamically generate interfaces can address the problem of not tailoring just to the content needed to serve the user, but the actual structure of the interaction over time. Yet, these new systems raise an unresolved question: when generative UI is beneficial, how can it be introduced to users without it corroding the user's mental model of how the application should work? This implies that the design pattern put in place for generative UI must manage the tradeoff between adaptivity and consistency (Bieniek et al., 2024).

To actually manage the tradeoff of leveraging generative UI, instead of having the entire interface be generated with AI, we could restrict it to ephemeral, so that task-scoped components appear only when needed and disappear when their use case has passed. Past work on ephemeral interfaces argues that the permanence in ubiquitous environments can contribute to cognitive overload as there are too many controls and artifacts present (Döring et al., 2013). Conversely, ephemerality can reduce the persistent clutter of artifacts and help with better inter-

actions for the user within a specific time frame and context. While this research was grounded in physical interaction design, it provides principles that can be transferred to digital systems. These ephemeral elements should be discoverable when needed, easy to dismiss, and designed to disappear without affecting the surrounding environment's knowledge. For generative UI, it suggests that ephemeral components could be a natural fit for LLMs as the system could generate the UI contextually in almost real time (Bieniek et al., 2024). However, it must be kept in mind how the component lifecycle should be carefully designed to prevent an unstable user experience.

Ephemeral generative components have been implemented in past research without a full interface replacement, but with a temporary UI as an intermediate layer for exploration and understanding. For agentic programming, research used ephemeral interfaces that sat between the prompt and code generation to improve observability and comprehension of LLM generated code. It helped developers steer their results without needing to inspect the raw code (Cheng et al., 2024). This shows how ephemeral UI can function as a way to help users with a workflow that lets them steer the outcome and iteratively refine stochastic tasks for exploring alternatives before taking concrete actions. However, this application of ephemeral UI was done in a narrow domain and intent. This leaves the question of how ephemeral patterns can generalise to everyday productivity applications where tasks are diverse. If not implemented properly it could result in a disruption of interruptions, context switching, or inconsistency of familiar workflow.

Taking all existing literature into account, it establishes three relevant and incomplete variables to this capstone. Firstly, fully generative UI experiences can improve the user experience and enjoyment of interacting with generative AI applications compared to only chat-only interaction (Ahmed & Imran, 2025; Leviathan et al., 2025; Tang et al., 2025). Secondly, deployments of dynamic interface generations in specific domains can address the shortcomings of problems in need of rule-based logic where needs are constantly changing and require tailoring (Tambi, 2020). Thirdly, research on ephemerality shows how temporary time-bound interaction artifacts can reduce cognitive overload by decreasing the amount of permanent UI accumulation for the user. Nonetheless, a clear gap remains at the intersection of limited guid-

ance on how to design *ephemeral, task-scoped generative UI components* that can improve existing applications while preserving a predictable user experience. Prior work does not articulate reproducible design patterns that define when generative components should appear, how their scope should be bounded with an exit that does not disrupt the users' mental model of the application, and how they should preserve a predictable user interface. This capstone addresses that gap by generalising ephemerality into a framework of component patterns to integrate into everyday applications.

1.3 Research Question and Contribution

The gap identified in the literature motivates the following research question:

> *How can ephemeral, task-scoped generative UI components be designed to improve existing applications without disrupting the user experience?*

To address this question, the thesis makes a set of connected contributions. First, it synthesises a **conceptual framework** that defines a taxonomy of ephemeral support roles, a four-stage component lifecycle, and a set of design principles that constrain how temporary generative components may be introduced into an otherwise stable interface. Second, it **operationalises that framework** through a constrained component-specification model that defines what kinds of temporary support may be generated and how they may be structured. Third, it implements a **validation pipeline and bounded overlay approach** that checks generated specifications against structural and scenario constraints before rendering them as temporary, dismissible interface layers that preserve continuity with the host application. Fourth, it builds an **instrumented comparative study artifact**: an experimental website that implements both a conventional baseline interface and an ephemeral condition governed by the framework's rules, with event logging and questionnaire capture built into the same system. Fifth, it evaluates the artifact through a **comparative within-subjects user study** that measures task completion, efficiency, interaction effort, perceived helpfulness, intrusiveness, and user control across both conditions. Together, these contributions move beyond the existing literature by offering not only a theoretical account of ephemeral generative UI but also a concrete operationalisation and empirical evidence of how it performs in a bounded productivity scenario.

1.4 Thesis Outline

The remainder of this thesis is organised as follows. Section 2 presents the research methodology, including the conceptual framework, the design of the artifact, and the evaluation plan. Section 3 describes the technical implementation of the experimental website and its support-generation pipeline. Section 4 reports the results of the user study across all behavioural and self-reported measures. Section 5 interprets the findings in relation to prior work and discusses the limitations and robustness of the evaluation. Section 6 outlines directions for future work, and Section 7 concludes.

2 Methodology

2.1 Research Design

This project adopts a Design Science Research (DSR) approach because its primary goal is not only to study an existing phenomenon, but to create and evaluate an artifact that addresses an identified design problem. The problem established in the literature is that current generative interfaces often fall at one of two extremes: either assistance is constrained to static interface structures, or entire experiences are dynamically generated in ways that can undermine predictability and continuity. In response, this project proposes a conceptual framework for ephemeral, task-scoped generative UI components intended to augment existing applications without disrupting user workflow. The methodological contribution therefore lies in developing a framework that translates ideas from prior work on ephemerality, generative UI, and interaction support into a form that can guide design decisions in a broader productivity context.

Following this DSR logic, the study proceeds in three connected stages. First, a conceptual framework is synthesised from the literature in order to define the taxonomy, lifecycle, and design principles of ephemeral interface components. Second, that framework is instantiated in a website artifact that serves as a research instrument, allowing the proposed ideas to be embodied in a controlled interactive environment. Third, the artifact is evaluated through a comparative user study that contrasts an original interface with an ephemeral condition on a single, bounded task. This sequence ensures that the evaluation is not detached from theory: the framework informs the design of the artifact, and the artifact in turn provides the basis for assessing whether ephemeral augmentation can improve task performance and user experience without introducing excessive intrusiveness.

2.2 Conceptual Framework

This study develops a conceptual framework for ephemeral, task-scoped generative UI components to guide both artifact design and evaluation. In one sentence, the framework specifies *which temporary support roles may appear*, *how those supports are triggered and withdrawn*,

and *which constraints keep them useful without destabilising the host interface*. Positioning the framework here in the methodology is important because it defines the concepts that the artifact operationalises and the criteria against which the two interface conditions are later compared.

The first element is a taxonomy of component roles. Rather than classifying components by visual form, such as whether they appear as a panel, popover, or inline element, the taxonomy classifies them by the function they serve within the workflow. Three roles are distinguished. *Interpretive support* helps users understand context, available options, outputs, or likely consequences before acting. *Refinement support* helps users steer or configure an intended action before commitment, for example by comparing alternatives or adjusting parameters. *Task-execution support* helps users complete a bounded subtask inside the surrounding application without replacing the broader workflow. Framing these roles as a taxonomy is methodologically useful because it makes explicit what kind of assistance is being introduced in each scenario and avoids treating ephemeral behaviour as a single undifferentiated interface style.

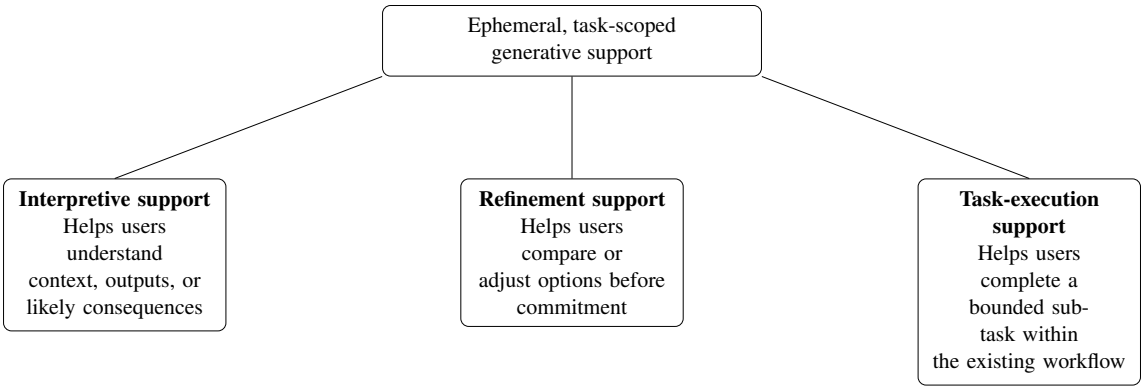


Figure 1: Taxonomy of ephemeral, task-scoped generative support roles.

The second element is a lifecycle describing how an ephemeral component enters and leaves use. Alternative stage patterns can be found in the literature, ranging from the minimal logic of temporary appearance and disappearance to more scaffolded models in which users configure support before committing to an outcome. For this study, the most appropriate synthesis is a four-stage lifecycle of *trigger*, *instantiation*, *interaction*, and *dismissal*. A component is triggered when a clear user goal or contextual need is detected that is not adequately supported by the persistent interface alone. That trigger may be explicit, such as a user request for help, or inferred from local behavioural and contextual signals, but only when the resulting sup-

port remains low-risk, relevant, and easy to dismiss. The component is then instantiated as a temporary interface element embedded within the existing application rather than replacing the surrounding interface. During interaction, users inspect, steer, or complete the immediate subtask while remaining oriented within the primary workflow. Finally, the component is dismissed once the task is completed, abandoned, or no longer contextually relevant. This lifecycle was selected because it retains the conceptual clarity of ephemerality while remaining concrete enough to implement as a state-based interaction model in the artifact.

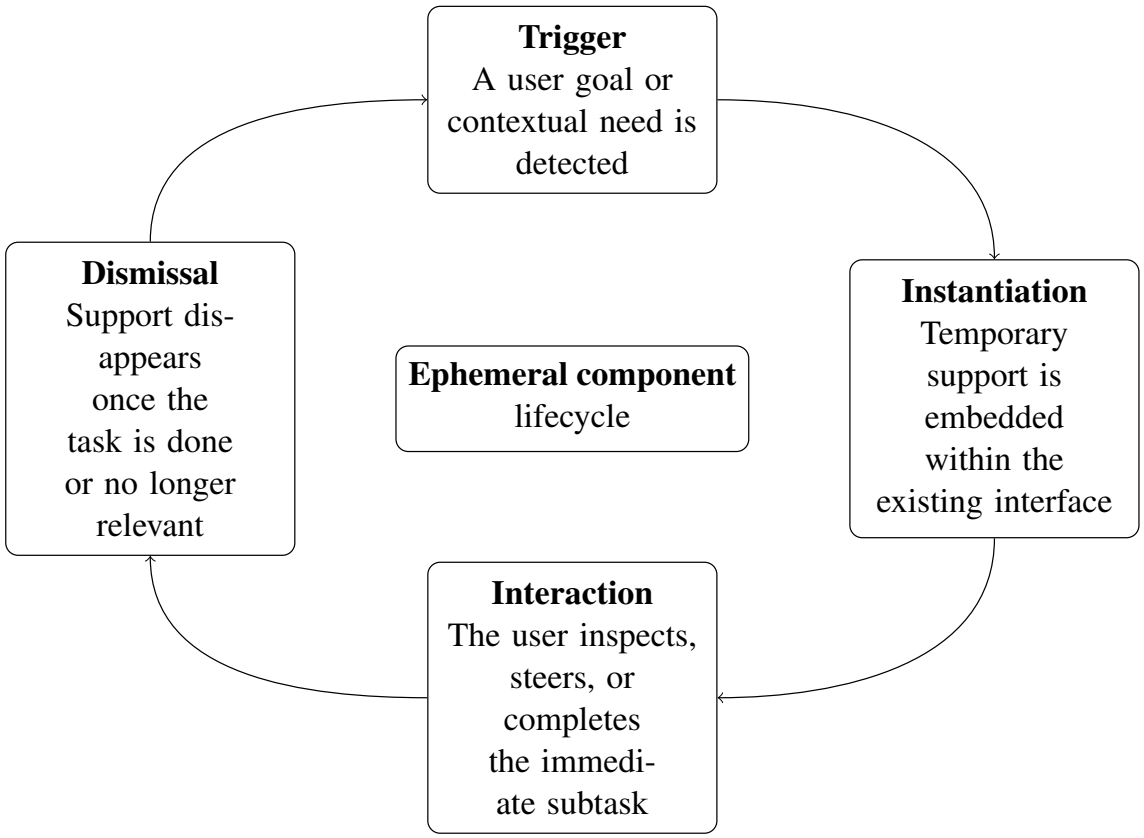


Figure 2: Lifecycle of an ephemeral, task-scoped generative UI component: trigger, instantiation, interaction, and dismissal.

The third element is a set of design principles that constrain how ephemeral components should be introduced into an otherwise stable interface. Five principles guide the framework. *Contextual relevance* means support should appear only when it serves a clear task-related purpose in the current situation. *Bounded scope* means support should remain limited to the immediate subtask rather than taking over the larger workflow. *Interface continuity* means the temporary component should preserve the surrounding page structure and the user's sense of place, rather than redirecting the interface toward an unrelated experience. *User control* means

users should be able to inspect, refine, confirm, or dismiss the support rather than being forced into a system decision. *Graceful disappearance* means the component should recede cleanly once it is no longer needed. When support is inferred rather than explicitly invoked, these principles impose an additional safety constraint: inferred assistance may suggest or preload options, but it should not perform destructive or strongly consequential actions without explicit user confirmation. Together, these principles translate the theoretical idea of ephemerality into practical design criteria that can be applied consistently during artifact development and later assessed in the evaluation.

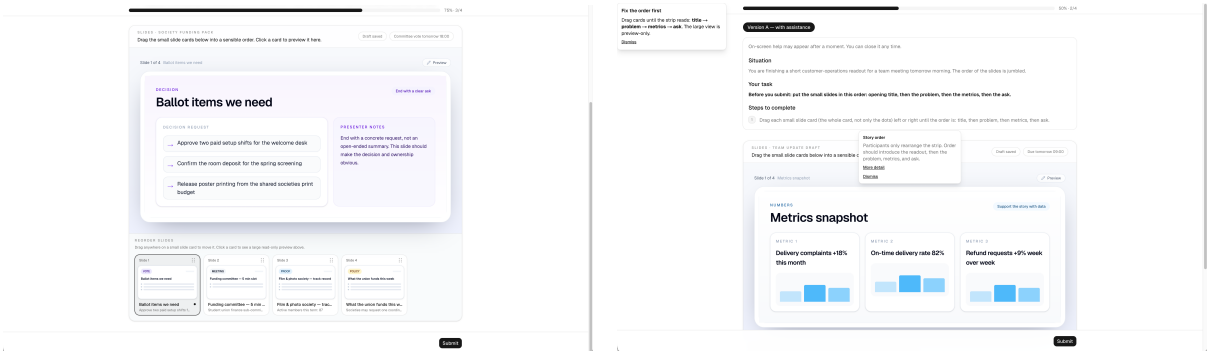
2.3 Artifact Development

The artifact developed in this project is a website used as a research instrument for evaluating the proposed framework in a controlled but realistic interaction setting. Rather than building a fully autonomous or fully generative application, the website is designed to preserve a recognisable baseline interface while allowing ephemeral, task-scoped components to be introduced at selected moments. This makes it possible to test whether the framework can augment an existing user experience without replacing the primary workflow altogether.

To operationalise the framework, the website will implement two interface conditions: an original condition representing the baseline experience and an ephemeral condition in which temporary, task-scoped components are introduced according to the taxonomy, lifecycle, and design principles defined above. The original condition will present the underlying tasks using a conventional persistent interface, whereas the ephemeral condition will surface temporary interface elements only when a user reaches a context in which additional guidance, refinement, or task support is needed. In this way, the comparison isolates the effect of ephemeral augmentation against the non-ephemeral alternative.

The website presents one scenario that represents a bounded productivity task within the application: a short presentation-refinement exercise foregrounding *refinement support*. Participants receive a small slide deck whose order has been intentionally scrambled and must restore it to a coherent narrative sequence before submitting. The interaction is deliberately narrow: the main slide canvas functions as a large read-only preview, while the participant performs the

task by reordering the slide thumbnails. The lifecycle of trigger, instantiation, interaction, and dismissal is implemented as the common interaction logic for both interface conditions. In the ephemeral condition, temporary support may propose order cues, comparisons, or lightweight explanations of narrative flow, but it does not edit slide content or take over the task. Success is therefore defined as submitting the deck in the predefined canonical order. Figure 3 shows the implemented study interface in both conditions.



(a) Baseline condition without temporary support.

(b) Ephemeral condition with temporary support visible.

Figure 3: Screenshots of the implemented experiment interface. In both conditions, participants complete the same slide-reordering task: the large slide canvas is a read-only preview, while the actual task action is performed through the thumbnail strip. The ephemeral condition preserves the same underlying task structure but adds temporary, dismissible support overlays.

The ephemeral condition relies on a constrained support-generation pipeline rather than unrestricted interface generation. When support is requested or triggered, the system produces a structured interface specification from the current task context, validates it against schema and safety constraints, and either renders it as a temporary overlay or falls back to a deterministic alternative. The technical details of this pipeline, including the specification model, the validation stages, and the rendering layer, are described in Section 3.

Although the ephemeral condition is allowed to be visually and structurally expressive, that expressiveness remains bounded by the framework. Temporary support may add task-relevant interface elements, annotate or highlight existing content, expose comparison views, or locally reconfigure a small region of the page. However, it does not replace the full application, arbitrarily reorganise the entire interface, or perform consequential actions without confirmation. In practice, the role category, lifecycle, task goals, and success criteria are fixed in advance, while local details such as wording, defaults, and the exact form of low-risk support may vary

in response to immediate context. The ephemeral version is therefore not intended to be wholly different for every participant. Instead, it is constrained adaptation: users encounter the same scenario structures and the same framework rules, but the temporary support may be parameterised slightly differently within those limits. This preserves analytical robustness while still allowing the artifact to reflect the broader motivation of adaptive, goal-relevant ephemeral interfaces.

2.4 Evaluation Design

2.4.1 Experimental Design

The framework will be evaluated through a comparative within-subjects user study using the website artifact described above. Each participant will complete two trials in total: the same slide-refinement scenario once as Version A and once as Version B. The presentation order of those versions is fixed, with Version A always shown first and Version B second, but the mapping between version label and experimental condition is randomised per participant so that one participant may receive the baseline first while another receives the ephemeral condition first under the alternative label. A within-subjects design is appropriate because it allows each participant to act as their own control, thereby reducing the influence of individual differences in digital literacy, task strategy, or prior familiarity with similar interfaces. This version-based counterbalancing preserves a consistent study flow while still varying which version corresponds to assistance across participants.

2.4.2 Participants

Participants will be recruited as regular users of web-based applications who can complete common digital tasks in a desktop browser environment. Recruitment will follow a convenience sampling approach appropriate to the scope of the capstone project. Inclusion criteria will focus on participants who are able to use standard web interfaces independently, while any exclusion criteria will be limited to factors that would prevent valid completion of the study tasks. The final sample size will depend on recruitment feasibility, but the study is designed to support repeated-measures comparisons between the two interface conditions.

2.4.3 Materials and Procedure

The primary study material is the experimental website, which implements both the original and ephemeral interface conditions. Participants will complete one bounded scenario twice: a presentation-style refinement task in which they reorder a short slide deck into the intended story sequence. The task structure, goals, and success criteria remain constant for all users so that the study remains comparable across sessions. Within that shared structure, the ephemeral condition may vary in how temporary support is presented so that the assistance can respond to local context while remaining constrained by predefined design rules derived from the framework. In other words, the study does not test unlimited personalisation or unrestricted interface takeover. It tests bounded interface adaptation in which temporary support may add, highlight, compare, or locally rearrange elements while preserving interface continuity and user control.

The procedure will be fully website-based. After opening the study website, participants will read the study information, provide consent, and complete a short background questionnaire about their comfort with websites and familiarity with AI-assisted tools. The website will then present two task trials in a fixed A-then-B sequence, representing the same scenario under the two experimental conditions, with the condition-to-version mapping randomised per participant. Before each trial, participants will receive a short task prompt that labels the interface version for fair debriefing. During each trial, the website will automatically log behavioural data. After each trial, participants will complete brief ratings on intrusiveness, perceived control, and helpfulness. After both trials are complete, participants will answer a final comparative questionnaire about overall preference between the two conditions, with an explicit reminder of which version corresponded to assistance for that participant. This fully instrumented procedure keeps the study structure consistent while allowing all behavioural and self-reported data to be collected in the same environment.

2.4.4 Measures

The evaluation combines behavioural and self-reported measures in order to capture both task performance and perceived experience. Behavioural measures are derived from interaction logs recorded by the website, while subjective measures are collected through short post-trial and

post-study questionnaires. The core claim being tested is not simply that ephemeral interfaces are preferred, but that they can improve task support without introducing additional disruption. To assess that claim, the study will measure efficiency through task completion time, effectiveness through task completion rate, friction through interaction count, and user experience through ratings of intrusiveness, perceived control, helpfulness, and overall preference. In addition, ephemeral-only interaction logs will record whether temporary support was triggered, requested, shown, used, dismissed, ignored, or expanded for further detail, allowing the study to interpret how participants actually engaged with the generated assistance.

Table 1: Dependent measures and collection methods.

Measure	Type	Collection	Analysis
Task completion rate	Binary	Event logging	McNemar's exact test
Task completion time	Continuous	Timestamps	Wilcoxon signed-rank test
Interaction count	Discrete	Event logging	Wilcoxon signed-rank test
Intrusiveness	Ordinal (1–7)	Post-trial Likert item	Wilcoxon signed-rank test
Perceived control	Ordinal (1–7)	Post-trial Likert item	Wilcoxon signed-rank test
Helpfulness	Ordinal (1–7)	Post-trial Likert item	Wilcoxon signed-rank test
Preference	Nominal	Final forced choice	Binomial test

2.4.5 Data Collection

Data collection will combine instrumented event logging with questionnaire responses. For each trial, the website will record task start and end times, completion outcomes, interaction counts, submitted task state, and the active interface condition. In the ephemeral condition, additional logs will capture support trigger events, explicit requests, support shown events, dismissals, usage confirmations, ignored support, detail expansions, and the generated support outputs themselves. Post-trial survey items will capture perceived intrusiveness, perceived control, and helpfulness, while the final questionnaire will capture overall preference between conditions. To link behavioural and survey data, each session will be associated with a participant identifier that is stored consistently across the website logs and questionnaire responses.

2.4.6 Analysis Plan

The primary analysis will compare the original and ephemeral conditions on each dependent measure using paired statistical tests selected for the data type, scale of measurement, and expected sample size.

Task completion rate is a paired binary outcome: each participant either completes the task correctly or does not, under each condition. This will be analysed using McNemar's exact test, which evaluates whether the proportion of participants who change completion status between conditions differs from what would be expected by chance. Completion proportions for each condition will be reported alongside the test result.

Task completion time and interaction count are continuous and discrete measures, respectively, that are paired across conditions. Because the expected sample size is modest and time data is typically right-skewed, both measures will be analysed using the Wilcoxon signed-rank test, a non-parametric alternative to the paired t-test that does not assume normality of the paired differences. Medians and interquartile ranges will be reported for each condition.

Post-trial Likert ratings for intrusiveness, perceived control, and helpfulness are ordinal measures on a seven-point scale. Each rating will be analysed using the Wilcoxon signed-rank test, which is appropriate for paired ordinal data. Medians will be reported rather than means to respect the ordinal nature of the scale.

Overall preference is a forced-choice measure in which each participant selects which condition they preferred. This will be analysed using a binomial test against the null hypothesis that preference is split equally between conditions. The count and proportion choosing each condition will be reported.

Ephemeral-only interaction logs will be analysed descriptively to show the proportion of participants for whom support was triggered, shown, used, dismissed, ignored, or expanded, which helps interpret whether a null or positive effect reflects the quality of the support itself or low engagement. The analysis will therefore focus on whether the ephemeral condition yields improvements in efficiency, effectiveness, or perceived helpfulness without increasing intrusiveness or reducing perceived control. Because the ephemeral experience may vary within bounded framework rules, the analysis treats that variation as part of the intended adaptive

behaviour of the artifact rather than as an uncontrolled change to the task itself.

3 Implementation

The previous section defined the conceptual framework, the artifact's purpose as a research instrument, and the evaluation design that structures the comparative study. This section describes how those ideas were translated into a working system. The goal is not to document every engineering decision, but to make the technical design legible enough that the reader can assess whether the artifact faithfully operationalises the framework and whether the instrumentation is sufficient to support the evaluation.

3.1 Architecture Overview

The artifact is built as a full-stack web application using the Next.js App Router with TypeScript (Vercel, 2026b). The server layer exposes a set of API routes that handle participant management, trial orchestration, event logging, questionnaire storage, and ephemeral support generation. Persistent data is stored in a PostgreSQL database accessed through Drizzle ORM, which provides type-safe queries and a migration-based schema workflow. The ephemeral support system relies on Google Gemini, accessed through the Vercel AI SDK, to produce structured interface specifications from the current task state (Google, 2026; Vercel, 2026a). The front end uses Tailwind CSS and a component library based on Radix UI primitives for consistent, accessible interface elements across both experimental conditions.

These choices reflect the requirements of a research instrument rather than a production application. Next.js (Vercel, 2026b) was selected because its App Router collocates server-side logic and client rendering in the same project, which simplifies the instrumentation of both request handling and user interaction. TypeScript and Zod provide compile-time and runtime type safety across the generation pipeline, which is critical when validating AI-generated output against a constrained specification. PostgreSQL and Drizzle offer relational integrity and migration tooling appropriate for structured experimental data. The Vercel AI SDK (Vercel, 2026a) abstracts provider-specific details while supporting structured output modes, which allows the generation step to request JSON that conforms to a predefined schema rather than parsing free-form text.

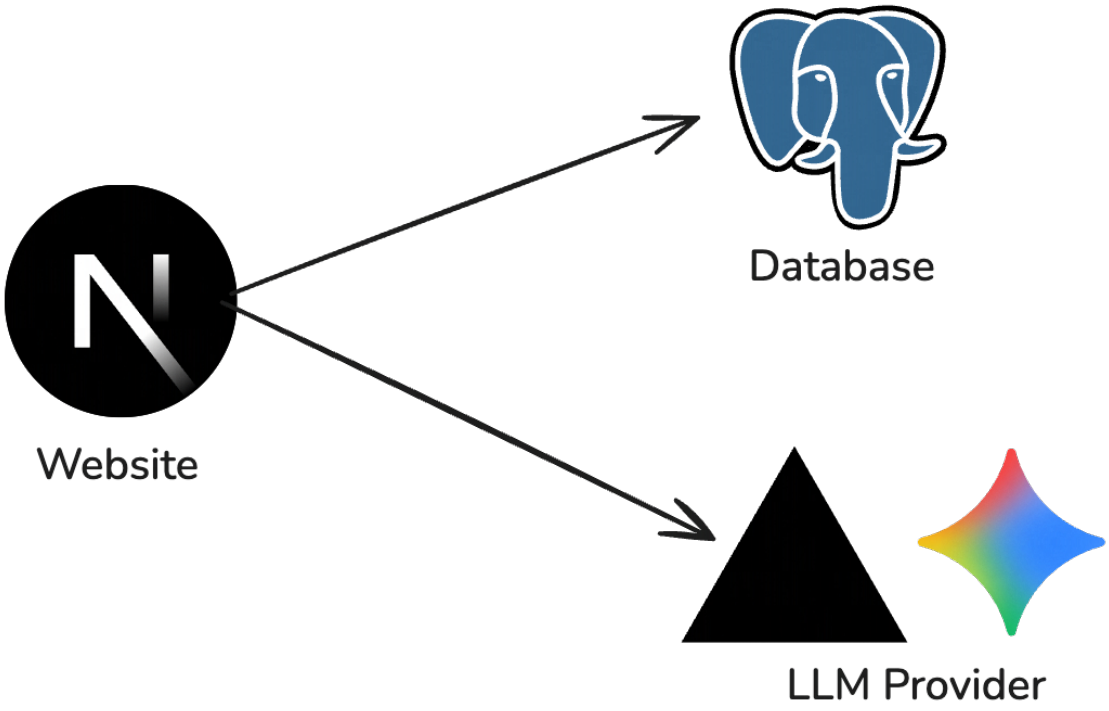


Figure 4: High-level system architecture of the experimental artifact.

3.2 Participant Flow and Session Management

The study procedure described in the methodology is implemented as a linear state machine that determines which page the participant should see at any point during the session. When a participant first opens the website, they are presented with the study information and consent form. Providing consent creates a participant record in the database and sets an HTTP-only session cookie that associates all subsequent requests with that record. No traditional authentication is required because the study collects no directly identifying personal data; the cookie serves only to link a browser session to its corresponding participant row.

After consenting, the participant completes a short background questionnaire covering age range, occupation, familiarity with web applications, and familiarity with AI-assisted tools. Submitting the background questionnaire acknowledges the study instructions and triggers the creation of the first trial row. From this point forward, a server-side state resolution function inspects the participant's record, their trial rows, and their questionnaire responses to determine the current study step. The possible steps form a fixed sequence: *background*, *study* (an active trial), *post-trial questionnaire*, *final questionnaire*, and *complete*. The client-side routing func-

tion maps each step to the corresponding page, so that refreshing the browser or navigating away always returns the participant to the correct point in the procedure.

Trial rows are created incrementally rather than all at once. The first trial is created when the participant acknowledges the instructions. Each subsequent trial is created only after the previous trial has been completed and its post-trial questionnaire has been submitted. This ensures that the participant cannot skip ahead or access later trials before finishing earlier ones.

Counterbalancing is implemented at the participant level. When a participant record is created, a boolean flag is randomly set to determine whether the baseline condition corresponds to Version A or Version B. The trial plan then assigns conditions to version labels using this flag, so that one participant may encounter the baseline first while another encounters the ephemeral condition first, both under the neutral label "Version A." The content variant for each condition is further diversified by hashing the participant identifier together with the scenario identifier, ensuring that the specific slide deck a participant sees in the baseline condition differs from the one they see in the ephemeral condition.

3.3 Data Model

The database schema consists of five tables designed to capture the behavioural and self-reported data required by the evaluation measures defined in the methodology. Figure 5 shows the entity–relationship structure: each participant may complete several trials; each trial may emit many logged events and many stored ephemeral support generations; questionnaire responses belong to a participant, and may optionally reference a trial (post-trial items) or omit a trial key (final questionnaire). Table 2 summarises what each table contributes to the evaluation. The `trial_events` table also stores `participant_id` for efficient querying, in addition to `trial_id`; the diagram omits that redundant edge for clarity.

Foreign key constraints with cascading deletion ensure referential integrity: deleting a participant removes all associated trials, events, questionnaire responses, and support outputs. Indexes on participant identifiers, trial identifiers, and event types support efficient querying during data export and analysis.

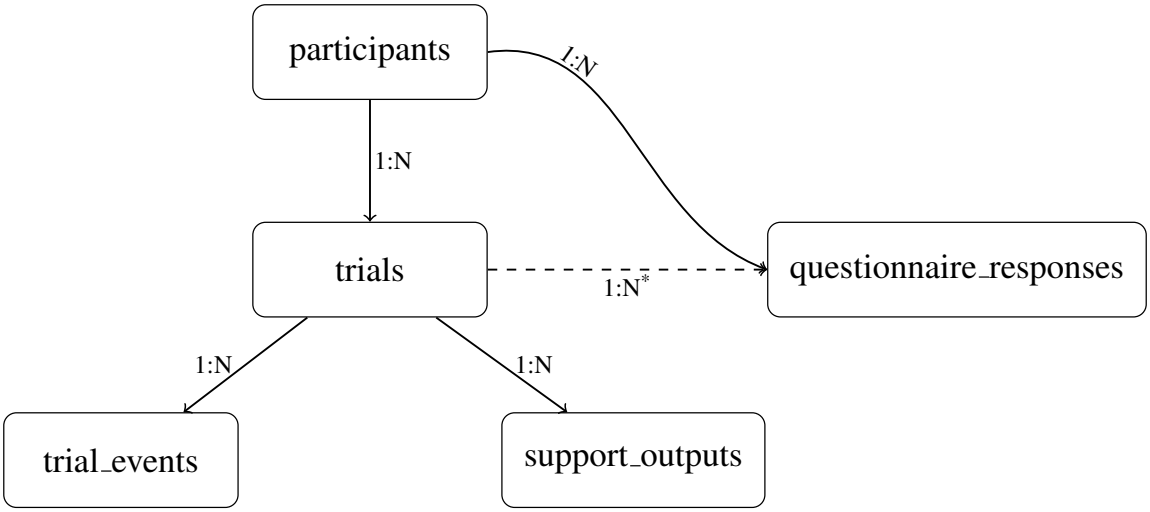


Figure 5: Entity–relationship diagram of the study database. Directed edges are labelled with cardinalities: one participant to many trials; one trial to many events and many support records; one participant to many questionnaire rows. The dashed edge indicates an optional foreign key: `trial_id` is `NULL` for final-questionnaire rows.

Table 2: Summary of each table's role relative to the evaluation measures. Full column definitions follow the Drizzle schema in the artifact.

Table	Role in the study
`participants`	Consent and background questionnaire; counterbalancing flag (`baseline_is_version_a`); links all rows for a session.
`trials`	One row per task trial: condition, scenario, timing, correctness, submitted answer, interaction count—supports performance and efficiency measures.
`trial_events`	Typed events with JSON payloads—supports interaction counts and ephemeral engagement (e.g. support shown, dismissed).
`questionnaire_responses`	Post-trial Likert items and final preference; `trial_id` set for post-trial rows, null for final rows—supports subjective measures.
`support_outputs`	Ephemeral condition only: model input state and output spec per generation—supports analysis of support quality and fallback.

3.4 Task Scenario Implementation

The study evaluates one scenario, the slide-reordering task described in the methodology, under two experimental conditions. The implementation uses a scenario registry that defines each scenario's metadata, including its taxonomy category, correct answer, allowed ephemeral targets, task prompts, and a function that builds the initial task state. This registry-based design means that adding a new scenario requires only a new registry entry rather than changes to the study flow or state machine.

The slide-reordering scenario presents participants with a short deck of four slides whose order has been intentionally scrambled. The main canvas area displays a read-only preview of the currently selected slide, while the actual task interaction occurs through a horizontal strip of draggable thumbnail cards. Drag-and-drop is implemented using the `@dnd-kit` library, which provides accessible keyboard and pointer-based reordering with sortable constraints. Participants reorder the strip and submit when they believe the sequence is correct.

Correctness is evaluated by comparing the submitted slide order against a predefined canonical order. Each scenario variant defines its own canonical sequence: variant A follows a corporate readout structure (title, problem, metrics, ask), while variant B follows a committee funding structure (meeting frame, policy, proof, ballot items). The variant assignment uses a deterministic hash of the participant identifier and scenario identifier to assign variant A or variant B to the baseline condition, with the opposite variant assigned to the ephemeral condition. This ensures that each participant encounters two different slide decks across the two trials, preventing familiarity effects from the task content while keeping the task structure identical.

Both conditions share the same task component, the same interaction mechanics, and the same submission logic. The only difference is whether the ephemeral support layer is active, which is controlled by the condition field on the trial row. This design isolates the effect of ephemeral augmentation from other variables.

3.5 Ephemeral Support System

The ephemeral support system is the core technical contribution of the artifact. It translates the framework's concepts of taxonomy, lifecycle, and design principles into a constrained generative pipeline that produces temporary interface overlays without replacing the host application. This subsection describes the three layers of the system: the specification model that defines what can be generated, the pipeline that produces and validates specifications, and the rendering layer that displays them.

3.5.1 Component Specification Model

Ephemeral support is represented as a typed tree structure called an `EphemeralSpec`. Each specification contains a version number, a root node, and metadata indicating whether the overlay is dismissible. Nodes are recursive: each node has a type, a property bag validated against a per-type schema, and optional children. The tree is constrained to a maximum depth of four levels and a maximum of six children per node. The decision to use JSON as the intermediate structure for these specifications was inspired by JSON Render, which similarly demonstrates how interface descriptions can be expressed declaratively and then rendered from structured data (Jason Lengstorf, 2026).

The component catalog defines seventeen component types, organised into five functional categories. Table 3 lists the types and their roles. Visual cue components draw attention to specific interface elements without adding content. Anchored annotation components attach textual or rich content near a target element. Relational components connect or compare two or more targets to support reasoning across elements. Layout containers position children within the viewport or relative to a target. Rich content components render sanitised HTML and inline SVG for more expressive explanations such as flow diagrams or annotated illustrations.

Each component type is associated with a Zod schema that validates its properties at runtime. These schemas enforce constraints such as maximum string lengths, numeric ranges, and required fields. For example, an `InspectPanel` requires a target identifier, a title of at most 120 characters, a summary of at most 280 characters, and optionally up to four detail lines of at most 200 characters each. These limits prevent the model from generating excessively long

Table 3: Ephemeral component catalog (version 3). Each type is validated against a per-component Zod schema that enforces field types, string lengths, and structural constraints.

Category	Component type	Role
Visual cues	`HighlightRing`	Amber ring around a target element
	`PulseRing`	Animated ring with configurable duration
	`ArrowCue`	Arrow pointing at a target
	`FocusMask`	Dims everything except the target
Anchored annotations	`AnchoredTooltip`	Plain-text tooltip near a target
	`HintStack`	Ordered list of short hints near a target
	`InspectPanel`	Expandable panel with summary and optional detail lines
	`ConsequenceNote`	Single italic consequence line near a target
	`SideNote`	Margin annotation beside a target
Relational	`ConnectorLine`	Dashed line between two targets
	`ComparisonStrip`	Short contrast text positioned under two targets
	`StepRail`	Numbered callouts across two to six targets
Layout containers	`Stack`	Vertical sequence container
	`ViewportPanel`	Absolutely positioned panel in viewport percentages
	`TargetOffsetPanel`	Panel positioned relative to a target with pixel offsets
Rich content	`FlowHtml`	Sanitised HTML block (only inside `ViewportPanel`)
	`AnchoredHtml`	Sanitised HTML anchored near a target

or unbounded output while leaving enough room for contextually relevant content.

The catalog is versioned so that changes to the available component types or their constraints can be tracked across study sessions. The current study uses catalog version 3.

3.5.2 Generation and Validation Pipeline

When ephemeral support is requested during a trial, the system executes a multi-stage pipeline that generates a specification, validates it against the catalog and scenario constraints, and either accepts it or falls back to a deterministic alternative. Figure 6 illustrates this pipeline.

The generation step constructs two prompts. The system prompt describes the specification schema, lists all available component types with their property schemas, states the allowed target identifiers for the current scenario, provides taxonomy-specific guidance (such as which components are most appropriate for refinement support), and enumerates the design rules that constrain output structure. The user prompt includes the current task state, including slide order, deck metadata, and review goals, together with an optional participant interaction snapshot that reflects the participant's progress at the moment support was requested. This snapshot allows the model to tailor its output to what the participant has already done rather than generating generic first-visit guidance.

The system sends these prompts to Google Gemini using the Vercel AI SDK's structured output mode (Google, 2026; Vercel, 2026a), which constrains the model to produce JSON conforming to a provided Zod schema. The schema used for model output is a simplified subset of the full specification schema, designed to avoid compatibility issues with the provider's structured output implementation while remaining strict enough to prevent most structural errors at generation time.

The model's output then passes through a series of server-side validation stages:

1. **Envelope parsing.** The top-level structure is checked for a valid version number, root node, and metadata.

2. **Node tree parsing.** The root and its descendants are recursively validated for correct node shape, child count limits, and maximum depth.

3. **Per-component property validation.** Each node's properties are validated against its type-specific Zod schema, enforcing field types, string lengths, and numeric ranges.

4. **Target allowlisting.** Every target identifier referenced in the specification is checked against the scenario's predefined list of allowed targets. Any reference to an interface element not declared in the scenario registry causes the specification to be rejected.

5. **Structural container rules.** Layout containers are checked to ensure they contain at least one child. This applies to `Stack`, `ViewportPanel`, and `TargetOffsetPanel`.

6. **Ancestry constraints.** `FlowHtml` nodes are permitted only within a `ViewportPanel` subtree, preventing free-floating rich HTML from appearing anywhere in the interface.

7. **HTML sanitisation.** Any HTML content in `FlowHtml`, `AnchoredHtml`, or `SideNote` nodes is passed through a server-side sanitisation step using DOMPurify with a strict allowlist of tags and attributes. This prevents injection of scripts, event handlers, or disallowed elements while permitting semantic HTML and inline SVG for diagrams.

If any stage rejects the output, the system falls back to a deterministic, hardcoded specification for the current scenario. Fallback specifications are defined per scenario and per variant, ensuring that every participant in the ephemeral condition receives some form of support regardless of model behaviour. The fallback specifications use the same component types and target identifiers as model-generated output, so they appear visually consistent with the rest of the ephemeral experience.

Whether the specification was model-generated or a fallback, the full input state, model name, and output are persisted to the `support_outputs` table. This logging enables post-study analysis of generation quality, fallback frequency, and the relationship between input context and output content.

To verify this implementation end-to-end outside the aggregate study analysis, the pipeline was also tested locally against the real `/api/support` route on 2026-04-04. Three consecutive hesitation-triggered requests for the `slides-outline-refine` scenario completed successfully and were persisted as `support_outputs` rows 101–103, all with `usedFallback =`

`false` and `modelName = gemini-2.0-flash`. The richest of these records (row 103) produced a `Stack`-rooted specification containing `StepRail`, `InspectPanel`, `ConnectorLine`, and `AnchoredTooltip` nodes, demonstrating that a non-trivial multi-component overlay could be generated, validated, saved to PostgreSQL, and rendered without falling back. A condensed excerpt of this local trace is included in Appendix A.3.

3.5.3 Rendering Layer

Accepted specifications are rendered as a temporary overlay above the existing application interface using a React portal. The overlay component receives the validated specification tree and recursively dispatches each node to its corresponding catalog component based on the node's type. This dispatch model means that the renderer does not need to know about the content of any particular specification; it only needs to map types to components and pass through validated properties.

Components that reference target elements, such as `HighlightRing`, `AnchoredTooltip`, or `ConnectorLine`, resolve their targets by looking up DOM elements by identifier. A custom hook observes the target element's bounding rectangle and updates the overlay position when the layout changes, for example when the browser is resized or when other elements shift. A viewport clamping utility ensures that anchored components remain visible even when their target is near the edge of the screen.

The overlay respects the lifecycle stage of dismissal defined in the framework. When the specification's metadata marks the overlay as dismissible, a close control is rendered that allows the participant to remove the support at any time. Dismissal fires a logged event so that the analysis can distinguish between support that was actively used and support that was dismissed without engagement. The overlay does not persist across page navigations or trial boundaries, which enforces the ephemerality principle at the session level.

3.6 Instrumentation and Logging

The evaluation design in the methodology specifies several behavioural and self-reported measures. The artifact implements a corresponding instrumentation layer that captures the data

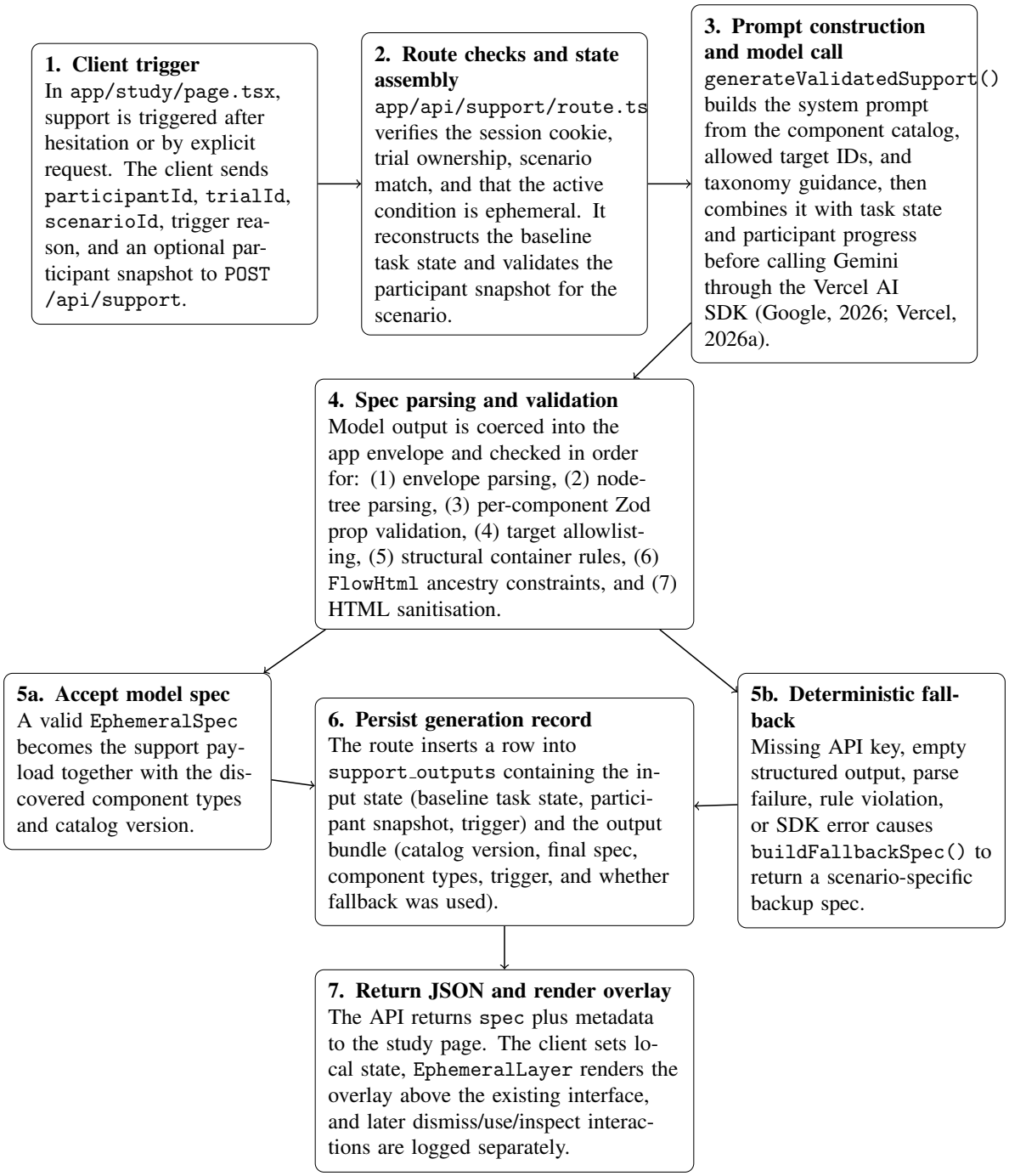


Figure 6: Ephemeral support generation and validation pipeline. Model output passes through seven validation stages before rendering; invalid output triggers a deterministic fallback. Both paths are logged for later analysis.

needed for each measure.

Client-side interaction events are sent to a logging API endpoint that inserts rows into the `trial_events` table. Each event includes a type label and a JSON payload with contextual details. A predefined set of event types is recognised as countable interactions, and the trial's running interaction count is incremented automatically when one of these events is recorded. Table 4 lists the event types and their relationship to the evaluation measures.

Table 4: Logged event types and their relationship to evaluation measures. Events marked with an asterisk (*) increment the trial's interaction count.

Event type	When recorded	Evaluation measure
`trial_viewed`*	Participant opens a trial	Interaction count
`slide_reordered`*	A slide is dragged to a new position	Interaction count
`answer_selected`*	An answer is first chosen	Interaction count
`answer_changed`*	An answer is revised	Interaction count
`support_requested`*	Participant explicitly requests support	Support engagement
`support_triggered`*	Support is triggered automatically	Support engagement
`support_shown`*	Support overlay becomes visible	Support engagement
`support_dismissed`*	Participant closes the overlay	Support engagement
`support_used`*	Participant acts on the support	Support engagement
`support_ignored`*	Support is present but not interacted with	Support engagement
`support_inspect_expanded`*	Participant expands an `InspectPanel` detail section	Support engagement

For the ephemeral condition, additional data is captured beyond interaction events. Each call to the support generation endpoint persists the full input state (task state and participant snapshot), the model name, and the complete output specification to the `support_outputs` table. This allows the analysis to examine not only whether support was shown and used, but also what the system produced and why, including whether a fallback was triggered.

Questionnaire responses are stored as individual question–response pairs rather than as monolithic form submissions. Post-trial ratings for intrusiveness, perceived control, and helpfulness are linked to the specific trial they follow, while final comparative questions are linked only to the participant. This structure supports direct joins between behavioural data and sub-

jective ratings at the trial level.

A data export endpoint produces a structured bundle of all study data, protected by a server-side secret. The export includes participant records, trial summaries with performance metrics, event logs, questionnaire responses, and support generation outputs, providing the complete dataset needed for the analysis plan described in the methodology.

4 Results

This section reports the outcomes of the comparative user study. All measures follow the analysis plan described in the methodology: behavioural data was extracted from the experiment database, and self-reported ratings were collected through post-trial and post-study questionnaires administered within the same website. Unless stated otherwise, paired comparisons use the Wilcoxon signed-rank test, medians and interquartile ranges (IQR) are reported for continuous and ordinal measures, and statistical significance is assessed at $\alpha = .05$.

4.1 Participants and Data Overview

A total of 48 participants accessed the study website. Of these, 23 completed both task trials and the final questionnaire and are included in the analysis. The remaining participants were excluded because they did not finish both conditions or did not complete the post-study questions. The reported results therefore describe the completer sample rather than all users who initially opened the study website. Table 5 summarises the background characteristics of the included sample.

Table 5: Participant background characteristics ($n = 23$).

Characteristic	Summary
Age range	19 (6); 20 (1); 21 (3); 22 (9); 23 (3); 24 (1)
Web application familiarity (1–7)	Mdn = 6.0, IQR = 6.0–7.0
AI tool familiarity (1–7)	Mdn = 6.0, IQR = 5.0–7.0
Condition order: baseline first	15 (65%)
Condition order: ephemeral first	8 (35%)

4.2 Task Completion Rate

Task completion was defined as submitting the slide deck in the correct canonical order. In the baseline condition, 14 of 23 participants (60.9%) completed the task correctly. In the ephemeral condition, 17 of 23 (73.9%) completed correctly. McNemar's exact test comparing discordant pairs yielded $p = 0.250$.

The ephemeral condition therefore showed a descriptively higher completion rate than the baseline, but the difference was not statistically reliable in the present sample.

4.3 Task Completion Time

Completion time was measured from the moment the participant started the trial to the moment they submitted their answer. Participants who completed the task in the baseline condition had a median time of 35.7 seconds (IQR: 16.4–73.4), compared to 56.1 seconds (IQR: 22.3–85.1) in the ephemeral condition. A Wilcoxon signed-rank test did not yield a statistically significant difference ($W = 125$, $p = 0.709$, $r = 0.094$). Although the ephemeral median was slightly higher, the result does not indicate a reliable difference in efficiency between conditions.

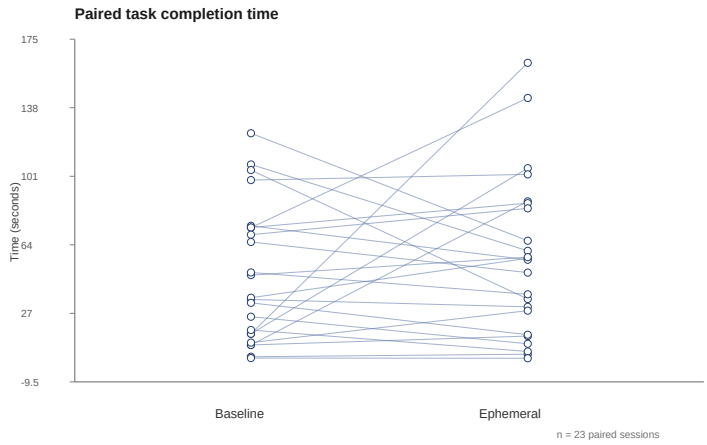


Figure 7: Paired comparison of task completion time (seconds) between the baseline and ephemeral conditions. Each line connects the same participant across conditions.

4.4 Interaction Count

The total number of logged interactions per trial was used as a proxy for task effort. The median interaction count in the baseline condition was 3.0 (IQR: 1.0–4.0), compared to 11.0 (IQR: 6.0–13.0) in the ephemeral condition. A Wilcoxon signed-rank test yielded a statistically significant difference ($W = 0$, $r = 1.000$; $p = <0.001$).

This higher count should be interpreted with caution because the ephemeral condition includes support-related events such as requesting support, support being shown, dismissal, and

detail expansion. The increase therefore reflects greater overall interface interaction, not only additional effort spent on the underlying slide-reordering task itself.

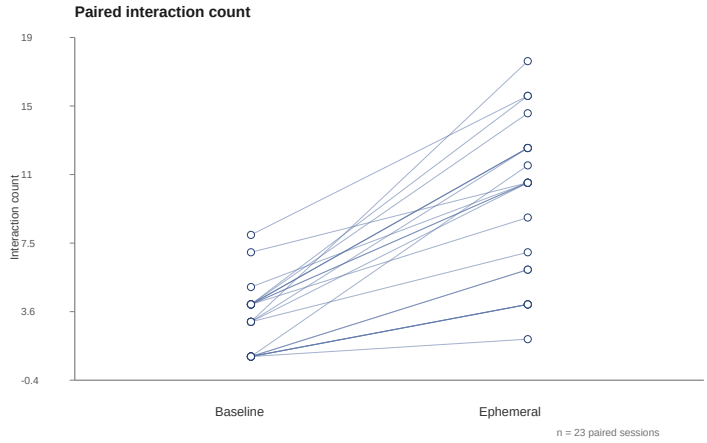


Figure 8: Paired comparison of interaction count between the baseline and ephemeral conditions.

4.5 Subjective Ratings

After each trial, participants rated the interface on three dimensions using a seven-point Likert scale. Table 6 summarises the results.

Table 6: Post-trial subjective ratings (7-point Likert scale, 1 = low, 7 = high).

Measure	Baseline		Ephemeral		Wilcoxon
	Mdn	IQR	Mdn	IQR	p
Helpfulness	5.0	3.0–6.0	6.0	5.0–6.5	0.029
Intrusiveness	3.0	1.5–5.0	5.0	4.0–6.0	0.004
Perceived control	6.0	4.5–7.0	6.0	4.5–6.0	0.579

The ephemeral condition received a higher median helpfulness rating than the baseline (6.0 vs. 5.0), and the Wilcoxon test indicated a statistically significant difference ($p = 0.029$). Within this sample, participants therefore rated the ephemeral interface as reliably more helpful.

Intrusiveness ratings were higher in the ephemeral condition (median 5.0) than in the baseline condition (median 3.0), and this difference was statistically significant ($p = 0.004$). Participants therefore experienced the temporary support as more noticeable or interruptive than the baseline interface.

Perceived control had the same median in both conditions (6.0 and 6.0), and the paired comparison was not significant ($p = 0.579$). Within the bounds of this study, the ephemeral support did not measurably reduce participants' sense of control over the task.

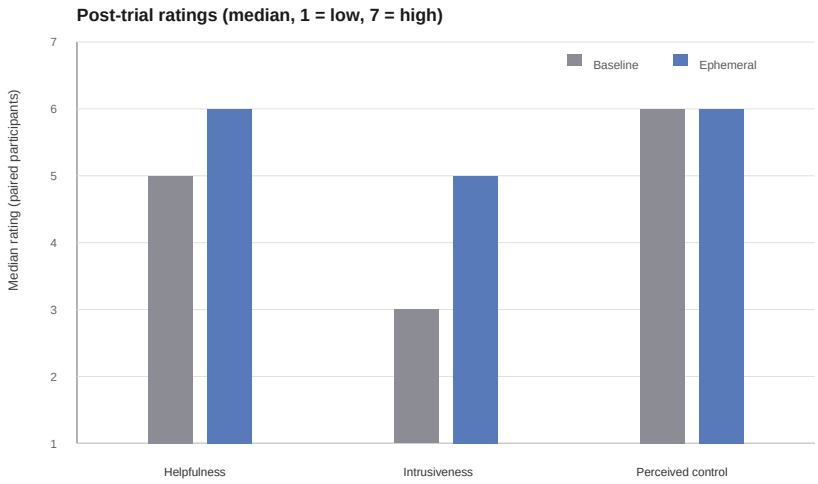


Figure 9: Median post-trial ratings by measure and condition (1 = low, 7 = high). Bars show the median rating across paired participants for each measure.

4.6 Overall Preference

After completing both trials, participants answered four forced-choice comparison questions. Responses were stored as Version A or Version B and mapped back to the actual experimental condition using each participant's counterbalancing assignment. Table 7 summarises the results.

Table 7: Final comparative preference questions ($n = 23$).

Question	Baseline	Ephemeral	No preference
Overall preference	6	11	6
More helpful	8	12	3
More intrusive	1	18	4
Real-life preference	9	10	4

Excluding the 6 participants who expressed no preference, 11 participants preferred the ephemeral condition and 6 preferred the baseline. A binomial test did not show a statistically

significant overall preference difference ($p = 0.332$), indicating that the sample did not meaningfully favour one condition over the other.

4.7 Ephemeral Support Engagement

To interpret the behavioural and subjective results in context, this subsection reports how participants actually interacted with the ephemeral support layer. These measures apply only to the ephemeral condition.

Of the 23 participants who completed the ephemeral trial, support was triggered or explicitly requested in 21 cases (91.3%). At the participant level, 6 participants recorded at least one `support_used` event, 21 recorded at least one `support_dismissed` event, 2 recorded at least one `support_ignored` event, and 3 recorded at least one detail-expansion event. These categories are not mutually exclusive: a participant could, for example, inspect or use support and later dismiss it in the same trial. The support generation pipeline produced 47 specifications in total, and no deterministic fallbacks were recorded (0 of 47, 0.0%).

This engagement pattern indicates that participants were usually exposed to the ephemeral support layer, but many chose not to engage deeply with it once it appeared. The condition-level outcomes should therefore be interpreted in light of partial uptake: the measured effects may reflect both the quality of the support itself and the fact that substantial portions of the offered assistance were dismissed after limited inspection or left unused altogether.

4.8 Summary of Findings

The ephemeral condition produced a higher descriptive task completion rate than the baseline, but the difference was not statistically significant. Completion time also did not differ significantly, with the ephemeral condition showing a slightly higher median time. Interaction count was substantially higher in the ephemeral condition, indicating greater overall interface activity during the trial. Subjective ratings suggested that the ephemeral interface was significantly more helpful, significantly more intrusive, and no different in perceived control. Final preference responses did not reveal a statistically reliable overall preference for either condition. Engagement data shows that support was usually surfaced, but sustained use was limited, which

provides important context for interpreting the mixed behavioural and subjective outcomes.

Table 8 provides an overview of all primary outcome measures.

Table 8: Summary of primary outcome measures across conditions.

Measure	Baseline	Ephemeral	p
Completion rate	60.9%	73.9%	0.250
Completion time (Mdn, s)	35.7	56.1	0.709
Interaction count (Mdn)	3.0	11.0	$<$0.001
Helpfulness (Mdn)	5.0	6.0	0.029
Intrusiveness (Mdn)	3.0	5.0	0.004
Perceived control (Mdn)	6.0	6.0	0.579
Overall preference	6 chose baseline; 11 chose ephemeral; 6 no preference		0.332

5 Discussion

This study examined whether ephemeral, task-scoped generative UI support can improve a bounded productivity task without disrupting the surrounding interface. The results point to a mixed answer. Relative to the baseline, the ephemeral condition produced a higher descriptive completion rate and a significantly higher helpfulness rating, while perceived control remained stable. However, it did not improve completion time or overall preference, and it significantly increased both interaction count and perceived intrusiveness. Taken together, these findings suggest that ephemeral support can be integrated into an existing interface without obviously undermining user control, but this particular implementation did not deliver clear performance gains without also introducing additional friction.

5.1 Interpretation and Relation to Prior Work

This framing positions the study somewhat differently from much of the recent discussion around generative UI. Prior work on generative interfaces has often emphasised adaptability, multimodality, and the capacity of AI systems to tailor interaction structures to user goals across contexts (Bieniek et al., 2024). That broader literature is useful for motivating why static interfaces may be insufficient in environments where user needs vary rapidly. At the same time, it also highlights a central design tension: the same adaptive power that makes generative UI attractive can also threaten predictability, continuity, and user control. The present study addresses that tension by evaluating a deliberately narrower alternative. Instead of replacing the interface or restructuring the entire workflow, the artifact constrains generative support to a temporary, local, and task-scoped layer. The results suggest that this bounded approach may offer more of an interpretive or confidence-supporting benefit than a pure efficiency benefit: participants rated the ephemeral condition as significantly more helpful, but they did not complete the task faster and did not show a statistically reliable increase in success.

The project also relates closely to BISCUIT, which showed that ephemeral UIs can act as an intermediate scaffold between user intent and LLM-generated output in computational notebooks (Cheng et al., 2024). In that study, the temporary interface helped users under-

stand generated code, explore alternatives, and reduce the burden of prompt engineering. The present work builds on that insight but shifts it into a different domain. Rather than supporting novice programmers in a notebook environment, the artifact evaluates ephemeral support in a bounded productivity scenario where the interface must preserve orientation inside an already recognisable application. The weaker and more mixed effects observed here suggest that the present implementation offered less interactive depth than BISCUIT's notebook scaffolds. Participants were often exposed to support, but only a minority actively used it, which implies that lightweight overlays may not produce strong benefits unless they offer enough immediate value to justify the extra attention they demand.

The discussion also connects to the earlier concept of ephemeral user interfaces proposed by Döring et al., who argue that intentionally temporary interface elements can reduce overload by limiting persistent accumulation and by presenting information only for the period in which it is relevant (Döring et al., 2013). Their work was grounded primarily in material and interaction-design perspectives rather than in web-based productivity systems, yet its core logic remains relevant here. The present thesis translates that logic into a digital software context by treating ephemerality not as a material property, but as a design constraint on interface behaviour. The subjective results highlight both sides of that design logic: perceived control remained stable, which supports the claim that the support stayed sufficiently bounded and did not take over the workflow, but intrusiveness increased significantly, showing that temporary appearance alone does not guarantee a low-disruption experience. In other words, bounded scope seems achievable, but trigger timing, presentation, and the immediate usefulness of the support remain critical if ephemerality is to feel genuinely unobtrusive.

Taken together, these comparisons suggest that the main value of the study is not to claim a final answer on generative UI in productivity tools, but to occupy an important middle ground in the literature. On one side are static interfaces that preserve continuity but may struggle to offer flexible, situation-specific assistance. On the other side are highly adaptive or fully generated interfaces that promise responsiveness but risk undermining user orientation and trust. The framework evaluated in this thesis proposes that ephemeral, task-scoped support may offer a compromise between these extremes: assistance can be adaptive, but only within predefined

boundaries concerning trigger conditions, scope, continuity, user control, and dismissal. The current results suggest that such support may be addable without major harm to perceived control, but they also indicate that the present intervention was too narrow, too intrusive, or too weakly taken up to demonstrate stronger performance benefits.

In practical terms, the findings suggest that ephemeral support is most promising in low-risk productivity contexts where bounded guidance can help users make progress without taking over the workflow. Examples include writing-oriented tasks such as drafting or revising a paper, lightweight task management such as checking off todos, or navigating a digital work environment where users need brief, contextual orientation. In these settings, temporary support can plausibly add value by helping users interpret local context or choose the next step while still leaving the underlying application stable and familiar. At the same time, the current results suggest that such support must remain optional, lightweight, and immediately useful; otherwise the cost of interruption may outweigh the benefit of assistance.

5.2 Limitations and Robustness

An important boundary of the study is the scope of the evaluation itself. The artifact was evaluated through a single presentation-refinement scenario that foregrounded refinement support rather than the full range of support roles proposed in the framework. This choice was appropriate for the capstone because it kept the procedure manageable and reduced participant fatigue. A design with three scenarios across two interface conditions would have required six task trials per participant, which would likely have increased drop-off, reduced engagement, and weakened the quality of responses. However, the consequence of this decision is that the findings cannot yet be generalised across multiple task types, domains, or support roles such as interpretive support or task-execution support.

The study is also limited by the breadth of interactions represented in the artifact. Some richer or more open-ended forms of support, such as more advanced writing assistance or more expressive temporary interaction patterns, were not retained in the final evaluation design. This means that the study primarily evaluates a bounded form of ephemeral support in a structured workflow rather than the broader design space of adaptive generative interfaces. As a result,

the conclusions are strongest for short, goal-directed refinement tasks and should not be treated as evidence that the same interaction logic would perform equally well in longer writing workflows, highly creative tasks, or more complex multi-step activities.

Further limitations concern sampling, device conditions, and duration. The study relies on a modest participant sample recruited within the practical constraints of the project, which limits the diversity of backgrounds, habits, and expectations represented in the data. In addition, the experimental artifact was not optimised for mobile use. If some participants accessed the study on smaller screens or in device conditions for which the interface was not well adapted, this may have introduced additional friction unrelated to the experimental condition itself and may therefore have skewed measures such as completion time, interaction effort, perceived intrusiveness, or overall preference. Participants also interacted with the system only within a short study session. This makes the evaluation suitable for examining immediate task performance and first impressions, but it does not capture longer-term effects such as habituation, changes in trust, or whether ephemeral assistance remains helpful once its novelty has worn off. The conclusions should therefore be read as evidence from a controlled short-term evaluation rather than as proof of long-term adoption in real-world settings.

Attrition also narrows the strength of the claims. Although 48 participants accessed the study website, only 23 completed both trials and the final questionnaire. This means the analysed sample likely overrepresents participants who were willing to continue through the full procedure, and the results should therefore be interpreted as evidence about completers rather than about everyone who initially encountered the study.

Another limitation is that the framework is tested through one particular implementation of the artifact rather than through multiple independent implementations. Positive or negative outcomes may therefore reflect not only the underlying framework, but also specific design decisions in this prototype, such as how support was phrased, when it was triggered, and how interaction options were presented. Subjective measures such as helpfulness, perceived control, and intrusiveness also rely on participant self-report and are therefore shaped by individual interpretation. These issues do not invalidate the findings, but they do mean that the results should be understood as an evaluation of a concrete operationalisation of the framework rather

than a definitive test of ephemeral support in the abstract.

The robustness of the paired comparison is also bounded by the sample size and by the structure of the engagement data. Condition order was randomised across participants, which helps reduce systematic ordering bias, but the final sample is too small to support strong claims about subtle order effects. In addition, the support-engagement metrics capture overlapping event types rather than a single exclusive participant state, so they are most informative as indicators of how often support was surfaced, interacted with, or dismissed, rather than as a complete behavioural taxonomy.

At the same time, several aspects of the research design strengthen the robustness of the study within these boundaries. Both interface conditions were evaluated on the same underlying task with the same success criteria, which supports a fair comparison between the baseline and ephemeral versions. The within-subjects design reduced the influence of individual differences because each participant encountered both conditions, while randomising trial order helped limit systematic ordering effects. The website also combined automated event logging with post-trial questionnaires, making it possible to compare behavioural outcomes and subjective experience rather than relying on anecdotes alone. Finally, although the ephemeral condition allowed bounded adaptation, that variation was constrained by predefined framework rules concerning scope, lifecycle, and user control, which preserved analytical comparability across sessions.

One unexpected aspect of the findings is that the behavioural and subjective measures do not all point in the same direction. The ephemeral condition was rated as significantly more helpful and achieved a slightly higher completion rate, yet it also produced no speed advantage and was experienced as more intrusive. This combination suggests that participants may have recognised the intent or potential value of the support without finding it seamless enough to outweigh its interaction cost during a short task. That tension is important because it implies that future refinements may need to focus less on adding support in principle and more on making that support feel timely, lightweight, and worth engaging with. Taken together, these limitations and robustness measures suggest that the study should be interpreted as a focused but credible first evaluation of ephemeral, task-scoped support in productivity software. The

results are sufficient to discuss the promise of the framework in a controlled setting, but they do not establish broad generalisability across contexts. Section 6 sets out concrete directions for extending scenarios, evaluation methods, and the design space of ephemeral components.

6 Future Work

The present evaluation is intentionally narrow: one scenario, one operationalisation of the framework, and a remote, self-paced procedure. The directions below follow from that scope and from what the artifact already suggests about where ephemeral, task-scoped support may be most informative next.

6.1 Broader scenarios and user populations

The study used a single presentation-refinement workflow. Ephemeral support is unlikely to matter equally in every domain. Prior work such as BISCUIT suggests that temporary interface scaffolds can work well in computational notebook settings associated with programming and data-science exploration, where users benefit from inspecting and steering intermediate output (Cheng et al., 2024). BISCUIT is therefore a useful contrast for the present thesis: it evaluates ephemeral support in a notebook-centred exploratory workflow, whereas this project examines whether similar temporary support can add value inside a more conventional productivity interface whose primary structure must remain stable and recognisable. In software development more broadly, however, many users already rely on mature patterns such as inline completion and tab-style acceptance flows in editors; in that setting, a bounded ephemeral layer may add less distinct value, or may need to compete with tooling that is already deeply integrated into the task. By contrast, everyday office or administrative work—where users must complete structured tasks inside interfaces they use only occasionally—may leave more room for lightweight, task-scoped assistance that does not assume expert shortcuts or a rich existing toolchain. This possibility also aligns with the broader claim in the generative-UI literature that adaptive support may be most valuable where conventional interfaces struggle to accommodate varying user needs across contexts (Bieniek et al., 2024; Tambi, 2020). A useful next step is therefore to evaluate the framework across several workflows and to sample participants with different levels of familiarity with the host application and with productivity software in general. Such work would test whether benefits cluster in contexts where users are less able to self-scaffold, and would clarify when ephemeral support is complementary rather than redundant relative to

established interaction models.

6.2 Deeper evaluation methods

Increasing the number of remote completions would improve statistical power, but it would not on its own address every open question about *why* participants behaved as they did, or how they interpreted ephemeral support moment to moment. A complementary direction is structured, in-depth evaluation: for example moderated sessions with think-aloud protocols, where a researcher can observe navigation problems, misunderstanding of triggers, and emotional reactions that web logs and post-task questionnaires only partially capture. Combining a controlled quantitative comparison with a smaller set of rich qualitative sessions would better connect design choices to observed friction and to subjective measures such as perceived control and intrusiveness.

6.3 Richer ephemeral component catalogues

The evaluation artifact deliberately used a constrained set of ephemeral UI components so that conditions remained comparable and the study remained feasible within the capstone timeline. The underlying framework could support a wider catalogue: more varied layouts, stronger personalisation of content and tone, and more expressive temporary interaction patterns than were deployed here. Exploring that wider design space would also connect more directly to the broader literature, which frames generative interfaces in terms of adaptive, context-sensitive interaction structures while also warning that flexibility can come at the cost of predictability and user control (Bieniek et al., 2024). At the same time, the foundational logic of ephemerality suggests that temporary interface elements should appear only when relevant and should recede without leaving persistent clutter (Döring et al., 2013). Expanding the catalogue therefore raises its own research questions—how to preserve predictability and user agency when surface behaviour varies more widely, and how to evaluate "wilder" or more personalised variants without confounding condition effects with implementation novelty. Future work could treat catalogue design as a first-class object of study: progressive disclosure of complexity, user-tunable boundaries for ephemerality, and systematic comparison of a small number of high-

diversity implementations against a stable baseline.

7 Conclusion

This thesis asked how ephemeral, task-scoped generative UI components can be designed to improve existing applications without disrupting the user experience. To address that question, it developed a conceptual framework for ephemeral support, implemented the framework in an experimental web artifact, and evaluated the artifact against a persistent baseline in a within-subjects user study. The evaluation provides a qualified answer: ephemeral support can be introduced in a bounded way that preserves users' sense of control, but the implementation tested here did not produce clear performance gains without also introducing additional intrusiveness.

Across the study measures, the ephemeral condition showed a slightly higher task completion rate, although that difference was not statistically significant, and a significantly higher perceived helpfulness rating. Completion time likewise did not improve. By contrast, interaction count increased substantially and perceived intrusiveness increased significantly, while perceived control remained stable and overall preference did not clearly favour either condition. These findings suggest that the main contribution of the ephemeral layer in this prototype was not faster task execution, but a clearer sense of perceived support that came with a noticeable interaction cost.

The thesis's individual contribution lies not only in evaluating ephemeral support as an idea, but in operationalising it through a constrained component-specification model, a validation pipeline for generated interface specifications, a bounded ephemeral overlay approach that preserves host-interface continuity, and an instrumented comparative study artifact that makes empirical evaluation possible.

The broader implication is that ephemeral, task-scoped generative UI remains a promising design direction, but its value depends on more than simply making support temporary. For such support to improve existing applications in practice, it must appear at the right moment, be immediately legible, and offer enough value that users choose to engage with it rather than dismiss it. The present findings suggest that the most credible application area is low-risk productivity work: tasks such as drafting or revising a paper, checking off todos, or briefly

helping users navigate their work environment are better matches than high-stakes or heavily interruption-sensitive workflows. The framework proposed in this thesis helps define those boundaries, while the study shows both their promise and their current limits. Future work should therefore test richer scenarios, broader user populations, and more refined support behaviours in order to determine when ephemeral generative assistance becomes not only possible within stable interfaces, but meaningfully beneficial.

References

Ahmed, A., & Imran, A. S. (2025). The role of large language models in ui/ux design: A systematic literature review. *arXiv preprint arXiv:2507.04469*. https://arxiv.org/abs/2507.04469

Bieniek, J., Rahouti, M., & Verma, D. C. (2024). Generative ai in multimodal user interfaces: Trends, challenges, and cross-platform adaptability. *arXiv preprint arXiv:2411.10234*. https://arxiv.org/pdf/2411.10234

Cheng, R., Barik, T., Leung, A., Hohman, F., & Nichols, J. (2024). Biscuit: Scaffolding llm-generated code with ephemeral uis in computational notebooks. *IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. https://arxiv.org/pdf/2404.07387

Döring, T., Sylvester, A., & Schmidt, A. (2013). Ephemeral user interfaces: Valuing the aesthetics of interface components that do not last. *ACM Interactions*, *20*(4), 32–37. https://interactions.acm.org/archive/view/july-august-2013/ephemeral-user-interfaces

Google. (2026). *Gemini api documentation*. Retrieved April 5, 2026, from https://ai.google.dev/gemini-api/docs

Jason Lengstorf. (2026). *Json render*. Retrieved April 5, 2026, from https://json-render.dev/

Leviathan, Y., Valevski, D., Kalman, M., Lumen, D., Segalis, E., Molad, E., Pasternak, S., Natchu, V., Nygaard, V., Venkatachar, S., Manyika, J., & Matias, Y. (2025). Generative ui: Llms are effective ui generators. *arXiv preprint arXiv:2410.11354*. https://generativeui.github.io/static/pdfs/paper.pdf

Tambi, V. K. (2020). Generative ai applications in customizing user experiences in banking apps. *The Research Journal (TRJ)*, *6*(6), 1–15.

Tang, F., Xu, H., Zhang, H., Chen, S., Wu, X., Shen, Y., Zhang, W., Hou, G., Tan, Z., Yan, Y., Song, K., Shao, J., Lu, W., Xiao, J., & Zhuang, Y. (2025). A survey on (m)llm-based gui agents. *arXiv preprint arXiv:2504.13865*. https://arxiv.org/abs/2504.13865

Vercel. (2026a). *Ai sdk*. Retrieved April 5, 2026, from https://sdk.vercel.ai/

Vercel. (2026b). *Next.js*. Retrieved April 5, 2026, from https://nextjs.org/

A Appendix

This appendix documents the key analysis steps used to transform the study database into the statistics, figures, and thesis-ready result values reported in Section 4. Because the examiner cannot assume access to the full repository, the appendix includes selected code excerpts that demonstrate the core logic of participant selection, statistical testing, figure generation, and LaTeX value generation. The excerpts are representative rather than exhaustive: they are intended to show how the analysis was implemented, not to reproduce every utility function in full.

A.1 Analysis Pipeline Overview

The analysis workflow was implemented as a small Bun-based pipeline. Rather than copying values manually into the thesis, the pipeline exported study data from PostgreSQL, computed the required descriptive and inferential statistics, generated the figure PDFs used in the results chapter, and wrote the LaTeX macro file that supplied numeric values to the manuscript. This structure reduced transcription risk and kept the reported values tied to executable analysis code.

The pipeline can be summarised in four stages:

1. export the required study data to intermediate CSV files,
2. compute the paired statistical tests used in the results chapter,
3. generate the PDF figures included in the results chapter, and
4. regenerate the LaTeX macros that populate reported values throughout Section 4.

This means that the values appearing in the results chapter were derived from the exported data rather than entered by hand. If the underlying database contents changed, rerunning the pipeline updated both the figures and the numeric placeholders used in the LaTeX source.

A.2 Representative Analysis Listings

The following listings were selected because they show the most thesis-relevant parts of the implementation: the rule used to determine the analysed sample, the paired non-parametric statistical test used for several outcomes, the generation of the main numeric figures, and the production of thesis-ready LaTeX commands.

A.2.1 Participant Inclusion Logic

Listing 1 shows the reusable SQL filter used throughout the analysis. A participant was included only if they completed both task trials and answered all four final comparison questions. This ensured that all reported measures were computed from a consistent completer sample.

```
WITH valid_participants AS (
  SELECT p.id, p.age_range, p.occupation,
         p.web_app_familiarity, p.ai_tool_familiarity,
         p.baseline_is_version_a
  FROM participants p
  WHERE (SELECT count(*) FROM trials t
         WHERE t.participant_id = p.id AND t.completed = true) = 2
    AND (SELECT count(DISTINCT question_key)
         FROM questionnaire_responses qr
         WHERE qr.participant_id = p.id
           AND qr.trial_id IS NULL
           AND qr.question_key IN (
             'final_preference',
             'final_helpfulness',
             'final_intrusiveness',
             'final_real_life'
           )) = 4
)
SELECT count(*) AS valid_completed
FROM valid_participants;
```

Listing 1: SQL logic used to define the valid participant sample.

A.2.2 Paired Statistical Testing

Listing 2 shows the central implementation of the Wilcoxon signed-rank test used for paired completion time, interaction count, and post-trial Likert outcomes. The function removes zero

differences, ranks the remaining paired differences by magnitude, computes the smaller rank sum W, and then uses an exact p-value for small samples or a normal approximation for larger ones. This matched the within-subjects design and the small paired sample available in the study.

A.2.3 Figure Generation

Listing 3 shows the core drawing loop used to produce the paired numeric plots for completion time and interaction count. Each participant contributes one baseline value and one ephemeral value, and the script draws a line connecting the two observations so that the within-participant change is visible directly in the figure.

A.2.4 Thesis-Ready Value Generation

Listing 4 shows how the final numerical outputs were converted into LaTeX commands. Instead of entering p-values, medians, and percentages manually into the thesis text, the script generated commands such as `\GenTimeP` and `\GenCompPctBaseline`. The results chapter then imported these commands directly, ensuring that the prose, tables, and summary values all used the same generated source.

A.3 Local Generation Trace

In addition to the aggregate study dataset analysed in Section 4, I captured a local end-to-end validation trace from the implemented support pipeline itself. This trace was taken from the real `/api/support` route rather than the development-only playground endpoint, so it exercised the same request handling, prompt construction, validation stages, PostgreSQL persistence, and client-facing response path used during the study. The selected example below is the locally persisted record with `support_output_id = 103`, captured on 2026-04-04 during a hesitation-triggered `slides-outline-refine` trial.

```
function wilcoxonSignedRank(
  baseline: number[],
  ephemeral: number[],
): { W: number; p: number; r: number; n: number } {
  const diffs: number[] = [];
  for (let i = 0; i < baseline.length; i++) {
    const d = ephemeral[i]! - baseline[i]!;
    if (d !== 0) diffs.push(d);
  }

  const n = diffs.length;
  if (n === 0) return { W: 0, p: 1, r: 0, n: 0 };

  const absDiffs = diffs.map((d) => ({ abs: Math.abs(d), sign: Math.sign(d)
  ↪  }));
  absDiffs.sort((a, b) => a.abs - b.abs);

  const ranks: number[] = [];
  let i = 0;
  while (i < absDiffs.length) {
    let j = i;
    while (j < absDiffs.length && absDiffs[j]!.abs === absDiffs[i]!.abs) j++;
    const avgRank = (i + 1 + j) / 2;
    for (let k = i; k < j; k++) ranks.push(avgRank);
    i = j;
  }

  let wPlus = 0;
  let wMinus = 0;
  for (let k = 0; k < n; k++) {
    if (absDiffs[k]!.sign > 0) wPlus += ranks[k]!;
    else wMinus += ranks[k]!;
  }

  const W = Math.min(wPlus, wMinus);
  const maxW = (n * (n + 1)) / 2;
  const p = n <= 25 ? wilcoxonExactP(W, n) : wilcoxonApproxP(W, n);
  const r = 1 - (2 * W) / maxW;

  return { W, p: Math.min(p, 1), r: Math.abs(r), n };
}
```

Listing 2: Core Wilcoxon signed-rank implementation used for paired outcomes.

```
const baselineVals = rows.map((r) => Number(r[opts.baselineKey]));
const ephemeralVals = rows.map((r) => Number(r[opts.ephemeralKey]));

for (let i = 0; i < rows.length; i++) {
  const b = baselineVals[i]!;
  const e = ephemeralVals[i]!;
  if (Number.isNaN(b) || Number.isNaN(e)) continue;
  const y1 = yToPdf(b, minY, maxY, chartBottom, chartTop);
  const y2 = yToPdf(e, minY, maxY, chartBottom, chartTop);
  page.drawLine({
    start: { x: xLeft, y: y1 },
    end: { x: xRight, y: y2 },
    thickness: 0.8,
    color: lineColor,
    opacity: 0.55,
  });
}

for (let i = 0; i < rows.length; i++) {
  const b = baselineVals[i]!;
  const e = ephemeralVals[i]!;
  if (Number.isNaN(b) || Number.isNaN(e)) continue;
  const y1 = yToPdf(b, minY, maxY, chartBottom, chartTop);
  const y2 = yToPdf(e, minY, maxY, chartBottom, chartTop);
  page.drawCircle({ x: xLeft, y: y1, size: 2.8, borderColor: pointColor });
  page.drawCircle({ x: xRight, y: y2, size: 2.8, borderColor: pointColor });
}
```

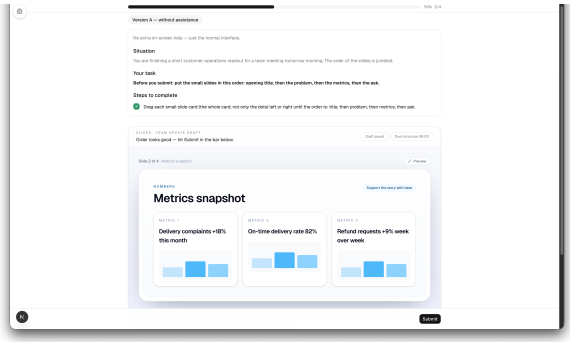
Listing 3: Core paired-plot drawing logic used for numeric result figures.

```
const lines: string[] = [
  "% AUTO-GENERATED by analysis/generate-results-tex.ts -- do not edit by
   ↪  hand.",
  "% Regenerate: bun run gen-tex",
  "",
];

cmd(lines, "GenNTotal", String(s.overview.totalAccessed));
cmd(lines, "GenNValid", String(v));
cmd(lines, "GenCompPctBaseline", s.completion.pctBaseline.toFixed(1));
cmd(lines, "GenCompPctEphemeral", s.completion.pctEphemeral.toFixed(1));
cmd(lines, "GenCompMcNemarP", fmtP(s.completion.mcnemarP));

const tw = s.time.wilcoxon;
cmd(lines, "GenTimeBaselineMdn", safeFixed(s.time.baselineMdn, 1));
cmd(lines, "GenTimeEphemeralMdn", safeFixed(s.time.ephemeralMdn, 1));
cmd(lines, "GenTimeP", fmtP(tw?.p ?? null));

writeFileSync(OUT_TEX, lines.join("\n") + "\n", "utf-8");
```

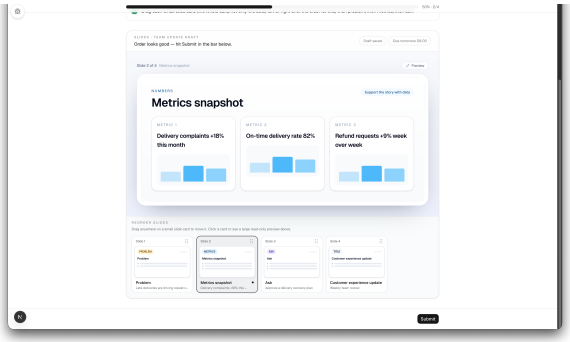
Listing 4: Generation of LaTeX commands used throughout the results chapter.

A.4 Participant Flow Screenshots

Figures 10–14 provide a chronological walkthrough of the participant-facing flow captured from the implemented artifact. These appendix figures complement the main-text interface screenshots by showing the sequence of states the participant moved through across the baseline trial, the ephemeral trial, and the final comparative questionnaire. One intervening transition capture that contained no visible interface content was omitted.



(a) Baseline trial with the plain interface visible and no temporary support.

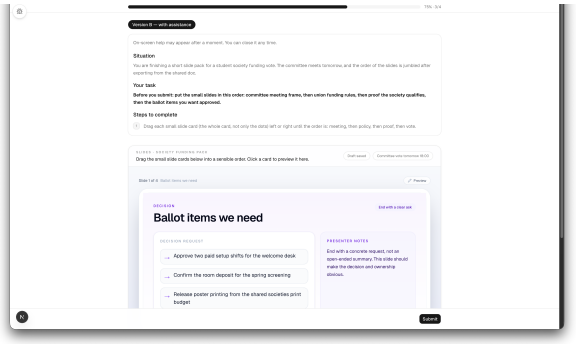


(b) Baseline task interaction while the participant reorders the slide strip.

Figure 10: Participant-flow walkthrough, part 1: baseline-condition task screens.

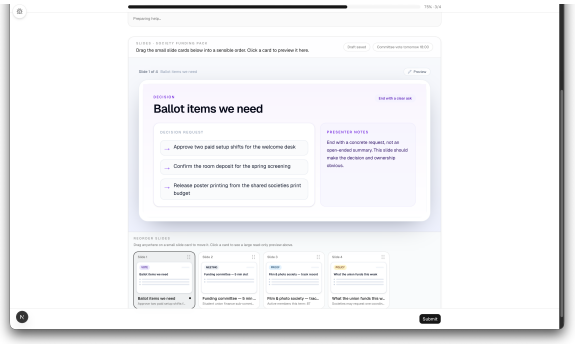
```
{
  "date": "2026-04-04",
  "route": "/api/support",
  "scenarioId": "slides-outline-refine",
  "participantId": "192571eb-71f5-4e69-9808-b1dedb71bdf6",
  "trialId": "f8523735-82a2-4926-9e7a-26a78eb13a19",
  "summary": {
    "successfulRuns": 3,
    "supportOutputIds": [101, 102, 103],
    "modelName": "gemini-2.0-flash",
    "allUsedFallback": false,
    "allProviderAttempted": true,
    "triggers": ["hesitation"],
    "observedComponentTypeSets": [
      ["Stack", "StepRail", "InspectPanel", "ConnectorLine"],
      ["Stack", "StepRail", "InspectPanel"],
      ["Stack", "StepRail", "InspectPanel", "ConnectorLine", "AnchoredTooltip"]
    ]
  },
  "selectedExample": {
    "requestId": "507d8f6e-525b-4478-8870-e9ac1be8e625",
    "supportOutputId": 103,
    "supportOutputCreatedAt": "2026-04-04T21:21:21.820Z",
    "trigger": "hesitation",
    "generation": {
      "modelName": "gemini-2.0-flash",
      "usedFallback": false,
      "fallbackReason": null,
      "providerAttempted": true,
      "catalogVersion": "catalog-v3",
      "componentTypes": [
        "Stack",
        "StepRail",
        "InspectPanel",
        "ConnectorLine",
        "AnchoredTooltip"
      ]
    },
    "inputExcerpt": {
      "participantSnapshot": {
        "scenarioId": "slides-outline-refine",
        "slideOrder": [
          "slide-b-meeting",
          "slide-b-policy",
          "slide-b-proof",
          "slide-b-vote"
        ]
      }
    },
    "persistedOutput": {
      "catalogVersion": "catalog-v3",
      "spec": {
        "version": 1,
        "root": {
          "type": "Stack",
          "props": {},
          "children": [
            {
              "type": "StepRail",
              "props": {
                "targetIds": [
                  "slide-b-meeting",
                  "slide-b-policy",
                  "slide-b-proof",
                  "slide-b-vote"
                ]
              }
            },
            {
              "type": "InspectPanel",
              "props": {
                "targetId": "deck-context-bar",
                "title": "Suggested Flow",
                "summary": "Try this order: meeting context -> relevant policy -> proof the society
                ↪  qualifies -> ballot-ready asks. Frame the vote first, then explain the rules, show why
                ↪  this society qualifies, and end with clear requests.",
                "details": [
                  "Opening with context sets the stage.",
                  "Highlighting policy grounds the story in the union's rules.",
                  "Evidence builds confidence in the society's track record.",
                  "Ending with concrete requests ensures a focused discussion."
                ]
```

(a) Baseline post-trial questionnaire immediately after task completion.

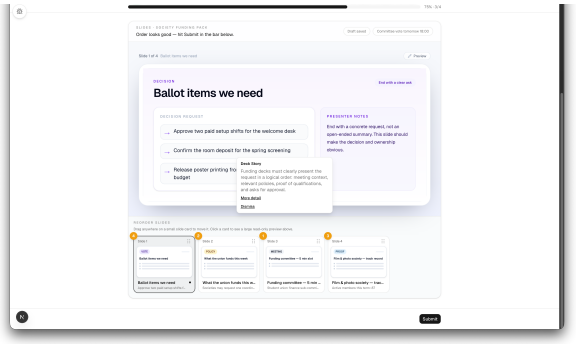


(b) Start of the ephemeral-condition trial, which warns that temporary help may appear.

Figure 11: Participant-flow walkthrough, part 2: transition from the baseline trial to the ephemeral trial.

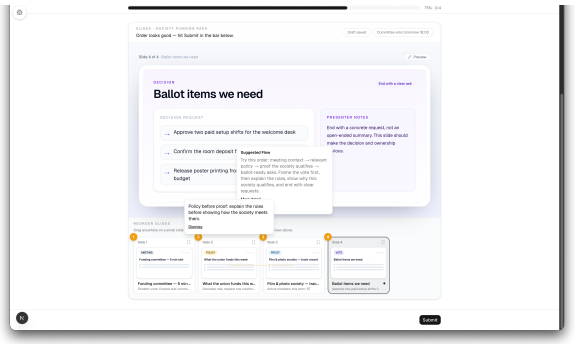


(a) Ephemeral-condition task before the support overlay expands.

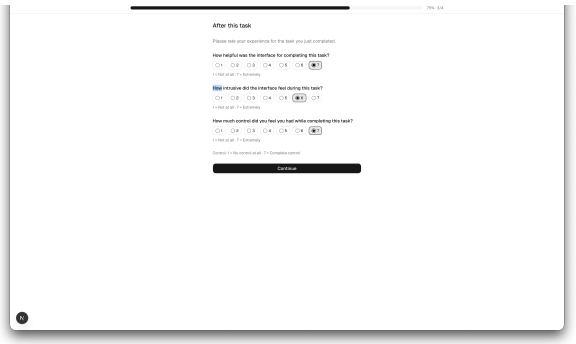


(b) Generated support visible as an anchored explanation and ordered markers over the slide strip.

Figure 12: Participant-flow walkthrough, part 3: ephemeral assistance becoming visible during the task.

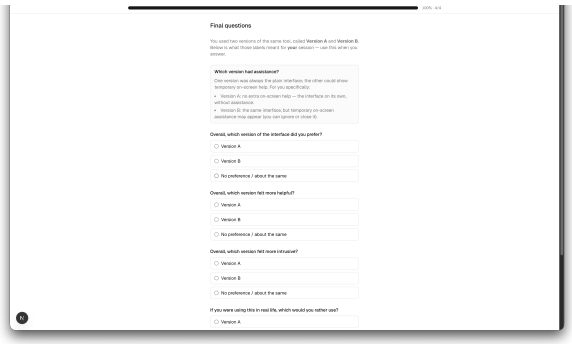


(a) Ephemeral overlay aligned to the current slide and reorder strip.

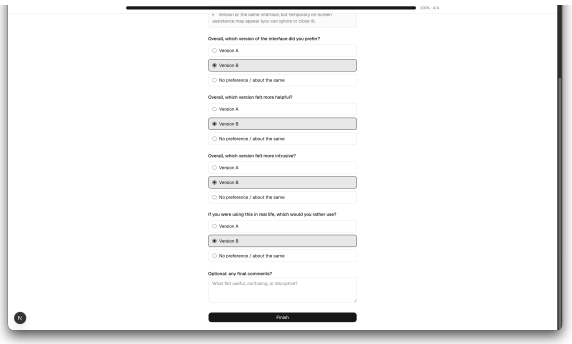


(b) Expanded support details combining an inspect panel, tooltip, and ordering cues.

Figure 13: Participant-flow walkthrough, part 4: richer support states within the ephemeral trial.



(a) Beginning of the final comparative questionnaire after both trials are complete.



(b) Completed final questionnaire with comparative judgments recorded.

Figure 14: Participant-flow walkthrough, part 5: final comparison questions at the end of the session.

A.5 Interpretive Note

These listings are sufficient to show the main route from raw study records to reported results: first the valid sample was defined consistently, then paired statistical tests were run, then publication-quality figures were generated, and finally the resulting numerical outputs were written into LaTeX commands consumed by the manuscript itself. This is the core evidence needed to demonstrate that the quantitative results chapter was generated through a reproducible analysis workflow rather than through manual transcription.